

Sistema ChatBot RAG con Voz para Procesamiento Inteligente de Documentos Científicos

Carlos Willian Luna Ccapa, Ronaldo Ticona Jancco, José Manuel Bustinza Quispe

Escuela Profesional de Ingeniería Informática y de Sistemas

Universidad Nacional de San Antonio Abad del Cusco

Cusco, Perú

210178@unsaac.edu.pe, 211816@unsaac.edu.pe, 224867@unsaac.edu.pe

Abstract—En los últimos años, la cantidad de documentos académicos y papers científicos ha crecido de forma exponencial, lo que hace que analizar, comprender y extraer información relevante sea una tarea compleja y demandante en tiempo. Este trabajo presenta un sistema ChatBot RAG con Voz que permite interactuar directamente con documentos PDF, haciendo preguntas en lenguaje natural y obteniendo respuestas contextualizadas. El sistema combina múltiples tecnologías avanzadas para procesar documentos académicos de manera eficiente, incluyendo un sistema RAG (Retrieval-Augmented Generation) que procesa documentos, los divide en fragmentos relevantes y realiza búsquedas semánticas para encontrar la información más pertinente. El sistema almacena embeddings localmente en ChromaDB, permitiendo consultas rápidas y preservando la privacidad de los documentos. La arquitectura del sistema incluye una interfaz moderna desarrollada con React, un backend con FastAPI y un motor RAG personalizado que implementa técnicas de procesamiento de lenguaje natural y aprendizaje automático para analizar, organizar y comprender textos académicos de manera automática.

Index Terms—procesamiento de documentos, modelos de lenguaje, minería de textos, recuperación de información, aprendizaje automático, inteligencia artificial, transformers, BERT, GPT, LayoutLM, extracción de información, análisis semántico, chatbots conversacionales, generación aumentada por recuperación, sistemas multimodales, procesamiento de lenguaje natural, documentos científicos, aprendizaje profundo, representaciones contextualizadas, arquitecturas neuronales, embeddings, RAG, base de datos vectorial, PyMuPDF, ChromaDB, SentenceTransformers, all-MiniLM-L6-v2, FastAPI, React, TypeScript, Vite

I. INTRODUCCIÓN

La producción científica crece de forma exponencial, lo que plantea el reto de diseñar sistemas capaces de procesar, comprender y organizar automáticamente la información contenida en los artículos académicos. Tradicionalmente, este problema se abordó desde la recuperación de información y el procesamiento de lenguaje natural (PLN). Sin embargo, en los últimos años, los avances en redes neuronales profundas y modelos de lenguaje de gran escala han abierto nuevas posibilidades.

En este contexto, surge la necesidad de desarrollar herramientas que permitan a los investigadores interactuar de manera más eficiente con documentos académicos. El sistema ChatBot RAG con Voz responde directamente a esta problemática al implementar una solución integral que va

mucho más allá de una simple integración con una API externa. La aplicación combina múltiples tecnologías avanzadas para procesar documentos académicos de manera eficiente, incluyendo las siguientes capacidades clave:

- **Procesamiento de documentos PDF:** Utilizamos PyMuPDF para extraer texto de manera precisa de documentos académicos complejos, manteniendo la estructura y contenido original.
- **Sistema RAG (Retrieval-Augmented Generation):** Implementado en `rag_system.py`, este sistema procesa documentos, los divide en fragmentos relevantes y realiza búsquedas semánticas para encontrar la información más pertinente.
- **Almacenamiento local:** A diferencia de soluciones que dependen completamente de servicios en la nube, nuestro sistema almacena embeddings localmente en ChromaDB, permitiendo consultas rápidas y preservando la privacidad de los documentos.

Los documentos académicos suelen ser extensos, técnicos y difíciles de consultar de forma rápida. Buscar información específica requiere leer muchas páginas y no siempre es eficiente. Este problema es precisamente lo que nuestro sistema resuelve mediante una arquitectura sofisticada que implementa división inteligente de documentos, indexación semántica y búsqueda contextual que comprende el significado de la consulta y encuentra fragmentos relevantes basados en similitud semántica.

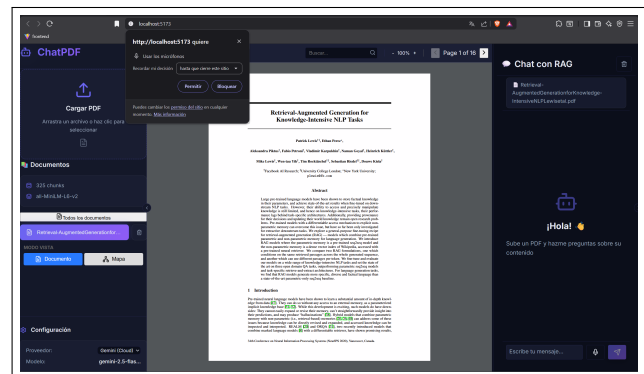


Fig. 1. Interfaz principal del sistema ChatBot RAG con Voz mostrando la interacción conversacional con documentos PDF.

II. ESTADO DEL ARTE

A. Fundamentos Teóricos y Algoritmos Clásicos

Los fundamentos del procesamiento de documentos científicos se apoyan en principios establecidos de la inteligencia artificial y la recuperación de información. Russell y Norvig [2] proporcionan el marco teórico fundamental para entender los sistemas inteligentes aplicados al procesamiento de documentos. Los algoritmos clásicos de recuperación de información descritos por Manning et al. [3] constituyen la base de los sistemas de búsqueda académica modernos, estableciendo métricas como TF-IDF y modelos probabilísticos que siguen siendo relevantes en aplicaciones contemporáneas.

El procesamiento de lenguaje natural, tal como lo describen Jurafsky y Martin [4], proporciona las técnicas fundamentales para el análisis sintáctico y semántico de textos científicos. Estos enfoques clásicos han evolucionado para incorporar representaciones distributivas de palabras que capturan mejor las relaciones semánticas en el dominio científico.

B. Minería y Extracción Estructural de Documentos

La extracción de información estructural de documentos científicos ha experimentado avances significativos. PDFFigures 2.0 [1] demostró ser una herramienta fundamental para la extracción automática de figuras, tablas y metadatos de papers académicos, estableciendo un estándar para la minería de elementos visuales en publicaciones científicas. Esta herramienta permite no solo extraer las figuras, sino también sus leyendas y referencias contextuales dentro del texto.

Con el auge del aprendizaje profundo, LayoutLM [6] revolucionó la comprensión de documentos al integrar la información visual y de posición del texto con representaciones semánticas. Este enfoque multimodal permite un análisis más preciso de la estructura documental, considerando tanto el contenido textual como la disposición espacial de los elementos.

C. Modelos de Lenguaje de Gran Escala

Los modelos de lenguaje han transformado radicalmente el procesamiento de documentos científicos. GPT-3, presentado por Brown et al. [5], demostró capacidades excepcionales en tareas de comprensión y generación de texto científico mediante aprendizaje few-shot, eliminando la necesidad de entrenamiento específico para muchas tareas de análisis documental.

La arquitectura Transformer, introducida por Vaswani et al. [7], estableció las bases para estos avances al proporcionar mecanismos de atención que permiten capturar dependencias a largo plazo en textos científicos. Radford et al. [9] desarrollaron GPT-2, demostrando el potencial de los modelos generativos en la comprensión contextual de documentos técnicos.

D. Modelos Especializados y Multilingües

BERT [8] introdujo el concepto de pre-entrenamiento bidireccional, mejorando significativamente la comprensión contextual en tareas de extracción de información científica.

Su arquitectura permite una mejor representación de conceptos técnicos y relaciones entre entidades en documentos académicos.

Los modelos multilingües como mT5 [10] han expandido las capacidades de procesamiento a documentos científicos en múltiples idiomas, facilitando el análisis de literatura internacional y la traducción automática de papers técnicos. Este enfoque es particularmente relevante para instituciones académicas que manejan literatura científica diversa.

E. Generación Aumentada por Recuperación

Lewis et al. [11] introdujeron el paradigma de Retrieval-Augmented Generation (RAG), que combina modelos de lenguaje generativos con sistemas de recuperación de información. Este enfoque es especialmente prometedor para el procesamiento de documentos científicos, ya que permite acceder a bases de conocimiento externas durante la generación de respuestas o resúmenes.

F. Aplicaciones en Sistemas Conversacionales

El desarrollo de chatbots inteligentes ha encontrado aplicaciones significativas en el procesamiento de documentos científicos. La revisión de literatura sobre chatbots en experiencia del cliente [12] proporciona insights sobre la implementación de sistemas conversacionales para la consulta de documentos académicos. Los estudios sobre chatbots como expertos en relaciones [13] sugieren aplicaciones en la recomendación de papers relacionados y en la facilitación de colaboraciones académicas. La seguridad de la información en chatbots [14] es crucial cuando estos sistemas manejan documentos científicos sensibles o propietarios.

G. Aplicaciones Educativas y de Aprendizaje

La efectividad de chatbots en el aprendizaje [15] ha demostrado potencial para crear sistemas tutoriales inteligentes basados en literatura científica. Estos sistemas pueden ayudar a estudiantes e investigadores a navegar y comprender documentos técnicos complejos. Los chatbots para promover actividad física y dieta saludable [16] ilustran cómo los sistemas de procesamiento de documentos pueden especializarse en dominios específicos, extrayendo y sintetizando información relevante de literatura médica y nutricional.

H. Integración en Entornos Educativos

La integración de chatbots en educación [17] mediante modelos como CHISM proporciona marcos teóricos para implementar sistemas de procesamiento documental en entornos académicos. Estos sistemas pueden automatizar la búsqueda bibliográfica y proporcionar respuestas contextualizadas basadas en corpus científicos específicos.

I. Representaciones Contextualizadas

Las representaciones de palabras contextualizadas [18] han mejorado significativamente la capacidad de los sistemas para entender terminología científica específica de dominio. Estas representaciones capturan mejor las variaciones semánticas de términos técnicos según el contexto científico en el que aparecen.

J. Sistemas de Embeddings y Bases de Datos Vectoriales

Los sistemas de embeddings han revolucionado la forma en que se procesa y recupera información en documentos científicos. El modelo `all-MiniLM-L6-v2` es un ejemplo de modelo eficiente y rápido que genera embeddings de alta calidad para tareas de similitud semántica. Estos embeddings permiten realizar búsquedas por similitud semántica en lugar de coincidencias exactas de palabras, lo que es crucial para encontrar información relevante incluso cuando la formulación es diferente.

Las bases de datos vectoriales como ChromaDB [20] permiten almacenar embeddings para realizar búsquedas por similitud semántica en lugar de coincidencias exactas de texto. La configuración de similitud coseno permite encontrar información relevante incluso cuando la formulación es diferente, lo que no sería posible con una simple búsqueda de palabras clave.

III. MODELO PROPUESTO

Nuestro aporte principal consiste en un sistema integral de ChatBot RAG con Voz que combina múltiples tecnologías avanzadas para procesar documentos académicos de manera eficiente. A diferencia de soluciones simples que se limitan a una integración con una API externa, nuestra arquitectura incluye componentes especializados que trabajan en conjunto para proporcionar una experiencia de usuario superior.

A. Motor RAG Personalizado

El archivo `rag_system.py` implementa un sistema completo de RAG que realiza las siguientes operaciones de manera integrada:

- Extrae texto de PDFs usando PyMuPDF con preservación de estructura
- Divide el texto en chunks relevantes de 500 palabras con solapamiento de 100 palabras
- Genera embeddings locales para cada fragmento usando el modelo `all-MiniLM-L6-v2`
- Almacena embeddings en ChromaDB para búsquedas rápidas
- Recupera fragmentos relevantes basados en similitud semántica
- Combina el contexto recuperado con la consulta del usuario

B. Sistema de Embeddings Local

Utilizamos el modelo `all-MiniLM-L6-v2` para generar embeddings locales, lo que proporciona las siguientes ventajas significativas:

- No enviamos documentos completos a servicios externos, preservando la privacidad
- Las búsquedas se realizan localmente con alta precisión
- El sistema puede operar offline después del procesamiento inicial
- Este modelo comprende el contenido semántico de los documentos de manera eficiente

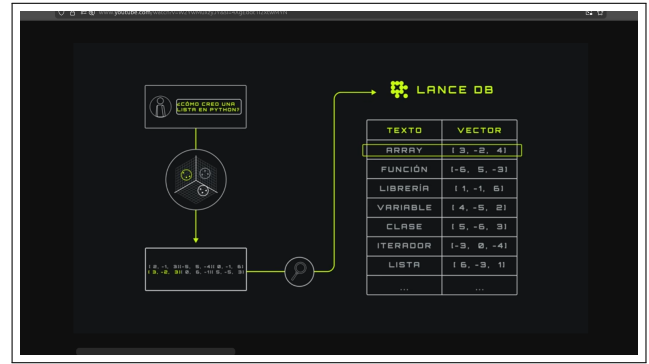


Fig. 2. Diagrama de arquitectura del sistema RAG mostrando el flujo de procesamiento desde el documento hasta la respuesta generada.

C. Integración con Múltiples Proveedores

El sistema puede utilizar tanto Gemini como modelos locales, demostrando flexibilidad arquitectónica. El proveedor se puede cambiar dinámicamente sin afectar el sistema RAG subyacente, lo que proporciona resiliencia y adaptabilidad.

D. Arquitectura Modular

La arquitectura del sistema se compone de los siguientes módulos especializados:

- **Interfaz de Usuario:** Desarrollada con React, TypeScript y Vite, proporcionando una experiencia de usuario rica y responsiva
- **Backend:** Implementado con FastAPI, que ofrece características como generación automática de documentación de API, validación automática de datos y alto rendimiento
- **Base de Datos Vectorial:** ChromaDB para almacenamiento y recuperación eficiente de embeddings
- **Sistema RAG:** Motor personalizado que implementa toda la lógica de procesamiento y recuperación de información

E. Funcionalidades Avanzadas

El sistema incluye funcionalidades especializadas que lo distinguen de soluciones convencionales:

- **Procesamiento de voz:** Capacidad para transcribir voz a texto y sintetizar respuestas en audio
- **Visualización de documentos:** Posibilidad de ver el documento original mientras se interactúa con él
- **Generación de mapas conceptuales:** El sistema puede generar representaciones visuales del contenido del documento
- **Síntesis automática:** Generación de resúmenes automatizados de documentos o secciones específicas
- **Búsqueda avanzada:** Búsquedas específicas dentro de documentos individuales o en todos los documentos procesados



Fig. 3. Visualización integrada de documentos PDF en la interfaz del sistema permitiendo lectura paralela durante la interacción conversacional.

IV. DISEÑO DEL APLICATIVO

La arquitectura del proyecto está diseñada como un sistema modular basado en el patrón **RAG (Retrieval-Augmented Generation)**. Su propósito es permitir que un Modelo de Lenguaje Grande (LLM) responda preguntas utilizando un cuerpo de conocimiento privado, conformado por documentos PDF cargados por el usuario. La arquitectura se organiza en tres capas lógicas principales: Interfaz, Orquestación y Sistema de Recuperación.

A. Capa 1: Interfaz de Usuario (Presentación)

Es la capa con la cual el usuario interactúa directamente. El proyecto ofrece una interfaz web moderna desarrollada con tecnologías avanzadas que garantizan una experiencia de usuario fluida y responsiva.

1) *Interfaz Web (React + TypeScript + Vite)*: La interfaz web está construida utilizando tecnologías modernas que garantizan rendimiento y mantenibilidad:

- **Framework**: React 19.2.0 con TypeScript 5.9.3 para desarrollo tipado seguro
- **Herramienta de construcción**: Vite 7.2.4 para un servidor de desarrollo rápido y optimización de producción
- **Componentes principales**:
 - ChatInterface para la interacción conversacional
 - PDFViewer para visualizar documentos directamente en la interfaz
 - ConceptMap para mostrar mapas conceptuales generados a partir de los PDFs
 - Iconos de Lucide React para mejorar la interfaz de usuario
- **Bibliotecas adicionales**:
 - React Markdown y Remark GFM para renderizar contenido con formato rico
 - Axios para hacer solicitudes a la API del backend
 - **ReactFlow y Dagre** para crear interfaces de diagramas y flujos de trabajo interactivos para la generación de mapas conceptuales

a) Generación de Mapas Conceptuales Interactivos:

Para la generación de gráficos interactivos (específicamente los Mapas Conceptuales), se utilizan las siguientes bibliotecas

en el frontend: Esta combinación de tecnologías permite representar visualmente las relaciones entre conceptos extraídos de los documentos científicos, facilitando la comprensión de estructuras complejas mediante una interfaz gráfica interactiva.

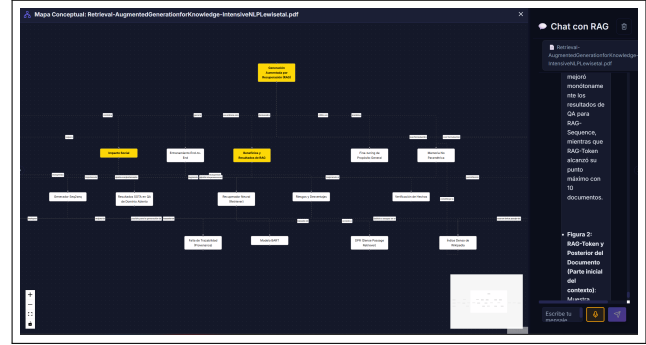


Fig. 4. Generador de mapas conceptuales interactivos mostrando la visualización de relaciones entre conceptos extraídos de documentos científicos usando ReactFlow y Dagre.

- **React Flow**: Biblioteca principal para renderizar el lienzo interactivo que permite:
 - Visualizar nodos y conexiones (edges) de forma dinámica
 - Interacción mediante zoom, desplazamiento y arrastre de nodos
 - Incluir componentes complementarios como Minimap, Controls y Background
- **Dagre**: Utilizado para el layout automático de los nodos. Como los mapas se generan dinámicamente con IA, Dagre calcula las posiciones (X, Y) de cada nodo para organizar el gráfico de forma jerárquica (de arriba hacia abajo) evitando superposiciones.
- **Lucide React**: Proporciona íconos decorativos que mejoran la experiencia visual en la interfaz del visor de mapas conceptuales.

Esta combinación de tecnologías permite representar visualmente las relaciones entre conceptos extraídos de los documentos científicos, facilitando la comprensión de estructuras complejas mediante una interfaz gráfica interactiva.

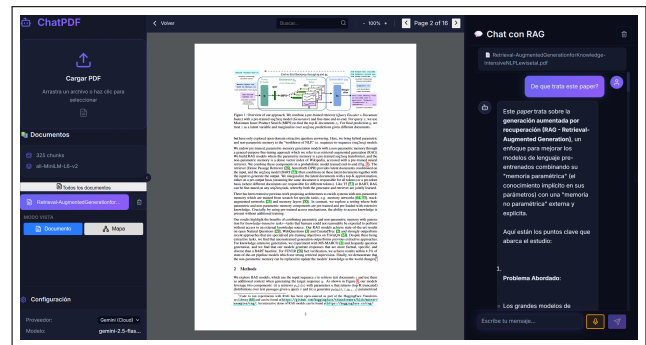


Fig. 5. Interfaz de usuario del sistema mostrando los componentes principales: chat conversacional, visor de PDF integrado y controles de interacción.

B. Capa 2: Orquestación y Lógica de Aplicación (ChatbotRAG)

Es el núcleo coordinador que conecta al usuario con el sistema RAG y con el LLM. Esta capa está implementada en el archivo `chatbot_rag.py` mediante la clase `ChatbotRAG`.

1) **Responsabilidades del Orquestador:** El orquestador cumple las siguientes funciones críticas:

- 1) **Gestor de Interacción:** Recibe preguntas desde la interfaz web y coordina las respuestas
- 2) **Ciente del LLM:** Usa la API tipo OpenAI conectada a un servidor local con el modelo Meta-Llama-3-8B-Instruct o la API de Google Gemini
- 3) **Coordinador RAG:** Solicita contexto relevante al `RAGSystem`
- 4) **Constructor de Prompts:** Combina instrucciones del sistema, contexto recuperado, historial conversacional y la pregunta actual
- 5) **Gestor de Historial:** Mantiene `self.messages` para conservar coherencia conversacional a lo largo de la sesión

TABLE I
COMPONENTES DEL SISTEMA RAG Y SUS FUNCIONES

Componente	Función Principal	Tecnología Utilizada
Procesador de Documentos	Extracción y división inteligente de texto con preservación de estructura	PyMuPDF (fitz)
Motor de Embeddings	Generación de representaciones vectoriales semánticas de 384 dimensiones	all-MiniLM-L6-v2
Base de Datos Vectorial	Almacenamiento persistente y recuperación eficiente por similitud	ChromaDB
Orquestador	Coordinación de componentes y gestión del flujo conversacional	ChatbotRAG

C. Capa 3: Sistema de Recuperación del Conocimiento (RAGSystem)

Es el núcleo funcional del sistema RAG que maneja ingesta, embeddings, almacenamiento y recuperación. Esta capa está implementada en el archivo `rag_system.py` mediante la clase `RAGSystem`.

1) **Procesador de Documentos:** El procesador de documentos realiza las siguientes operaciones:

- Extrae texto usando `PyMuPDF (fitz)` con preservación de estructura
- Segmenta en chunks de 500 palabras con solapamiento de 100 para mantener contexto
- Normaliza y limpia el texto antes de generar embeddings

2) **Motor de Embeddings:** El motor de embeddings utiliza tecnología avanzada para representación vectorial:

- Modelo: `all-MiniLM-L6-v2` de `SentenceTransformers`

- Convierte texto y preguntas en vectores numéricos de 384 dimensiones
- Implementa similitud semántica para la recuperación de información relevante

3) **Base de Datos Vectorial:** La base de datos vectorial proporciona almacenamiento y recuperación eficiente:

- Tecnología: `ChromaDB`
- Guarda embeddings y chunks de texto con metadatos asociados
- Utiliza búsqueda por similitud de coseno para encontrar fragmentos relevantes
- Mantiene colección global y colecciones especializadas por documento

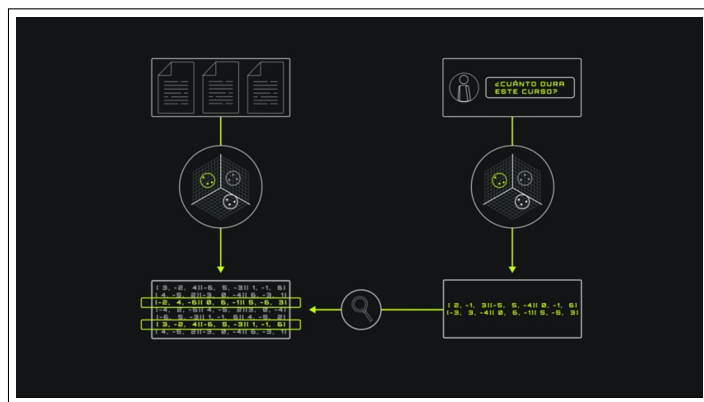


Fig. 6. Pipeline completo del sistema RAG mostrando el flujo desde el documento original hasta la generación de respuestas contextualizadas.

D. Tecnologías del Backend

El backend del sistema está implementado con tecnologías modernas y eficientes que garantizan alto rendimiento y escalabilidad.

1) **Framework Web:**

- **FastAPI 0.115.6:** Framework moderno y rápido para construir APIs con Python, basado en estándares abiertos (type hints de Python y OpenAPI)
- **Uvicorn 0.34.0:** Servidor ASGI de alto rendimiento para Python, utilizado para ejecutar aplicaciones FastAPI

2) **Procesamiento de Archivos:**

- **PyMuPDF (fitz) 1.24.10:** Biblioteca para manipular documentos PDF con extracción precisa de texto
- **python-multipart 0.0.20:** Permite manejar datos multipart/form-data para la subida de archivos

3) **Inteligencia Artificial:**

- **Google Generative AI SDK 0.8.5:** SDK oficial de Google para interactuar con modelos de IA como Gemini
- **OpenAI Python Client 1.58.1:** Cliente oficial de OpenAI para conectarse a modelos locales
- **SentenceTransformers:** Para generar embeddings semánticos de alta calidad

E. Tecnologías de Base de Datos

1) *Base de Datos Vectorial*: La base de datos vectorial utilizada proporciona las siguientes características:

- **ChromaDB 0.6.3**: Base de datos vectorial de código abierto diseñada específicamente para aplicaciones de búsqueda semántica y sistemas RAG
- **Algoritmo de similitud**: Cosine similarity (hnsw:space: cosine) para búsquedas eficientes
- **Ruta de almacenamiento**: ./chroma_db (por defecto) con persistencia local

V. ALGORITMOS DETALLADOS

En esta sección se describen los dos procesos fundamentales que permiten el funcionamiento del sistema: la generación de **embeddings** y el pipeline **RAG (Retrieval-Augmented Generation)**. Estos algoritmos constituyen el núcleo de la arquitectura, ya que permiten transformar documentos en vectores numéricos y, posteriormente, recuperar información relevante para generar respuestas enriquecidas.

A. Algoritmo 1: Generación de Embeddings

Este algoritmo transforma texto bruto en representaciones vectoriales de alta dimensionalidad que capturan semántica. Dichos vectores se almacenan en la base de datos para permitir búsquedas por similitud.

1) Entradas del Algoritmo:

- Texto procesado o segmentos de documento (*chunks*)
- Modelo de embeddings previamente cargado (all-MiniLM-L6-v2)

2) Pasos del Algoritmo:

1) Normalización del texto:

- Conversión a minúsculas para uniformidad
- Eliminación de caracteres irrelevantes y ruido
- Opcional: lematización o limpieza adicional según el dominio

2) Vectorización con modelo de embeddings:

- Se llama al modelo mediante `model.encode(texto)`
- El resultado es un vector $v \in \mathbb{R}^{384}$ que captura el significado semántico

3) Serialización y almacenamiento:

- El vector se convierte a lista con `.tolist()`
- Se almacena en la base vectorial junto con:
 - id único del chunk para referencia
 - Texto original del chunk
 - Metadatos (nombre del archivo, ruta, página, fecha de indexación)

3) *Implementación en Código*: La siguiente implementación muestra la función de generación de embeddings:

```
1 def generate_embedding(self, text_chunk):
2     """
3     Genera el embedding vectorial de un fragmento de
4     texto.
5
6     Args:
7         text_chunk: Fragmento de texto a vectorizar
8
9     Returns:
10        Lista con el embedding de 384 dimensiones
11    """
12    embedding = self.embedding_model.encode(
13        text_chunk).tolist()
14    return embedding
```

Listing 1. Función de generación de embeddings en el sistema RAG

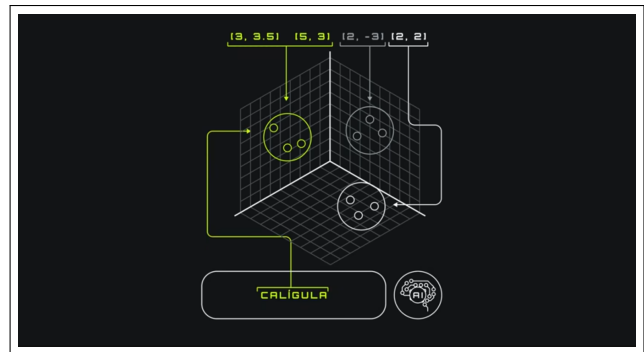


Fig. 7. Proceso de generación de embeddings mostrando la transformación de texto en representaciones vectoriales de 384 dimensiones.

B. Algoritmo 2: Pipeline RAG (Recuperación y Generación)

El objetivo del RAG es utilizar los embeddings previamente creados para recuperar información relevante y generar una respuesta fundamentada en el contenido de los documentos.

1) Entradas del Algoritmo:

- Consulta en lenguaje natural realizada por el usuario
- Base de datos vectorial ya indexada con los documentos procesados

2) Pasos del Algoritmo:

1) Vectorización de la consulta:

- La pregunta se convierte en un embedding usando el mismo modelo all-MiniLM-L6-v2

2) Búsqueda por similitud:

- Se ejecuta una consulta vectorial con métrica de coseno en ChromaDB
- Se recuperan los k chunks más cercanos en el espacio vectorial (por defecto $k = 3$)

3) Construcción del contexto:

- Los chunks recuperados se concatenan para formar el contexto documental

4) Construcción del prompt aumentado:

- Instrucciones del sistema para guiar al modelo
- Contexto generado a partir de los documentos relevantes
- Pregunta del usuario

5) Generación con LLM:

- El LLM recibe el prompt estructurado
- Produce una respuesta respaldada por el contenido del contexto

6) Salida final:

- La respuesta se devuelve al usuario
- Se almacena en el historial conversacional para mantener coherencia

3) *Implementación en Código:* La siguiente implementación muestra la función de recuperación de contexto RAG:

```
1 def get_context(self, query, k=3):
2     """
3     Recupera contexto relevante de la base de datos
4     vectorial.
5
6     Args:
7         query: Pregunta del usuario
8         k: Numero de fragmentos a recuperar
9
10    Returns:
11        Contexto formateado con los fragmentos
12        relevantes
13    """
14    # Generar embedding de la consulta
15    query_embedding = self.embedding_model.encode(
16        query).tolist()
17
18    # Buscar en la base de datos vectorial
19    results = self.collection.query(
20        query_embeddings=query_embedding,
21        n_results=k
22    )
23
24    # Extraer documentos recuperados
25    docs = results["documents"][0] if results["documents"] else []
26
27    if not docs:
28        return "No hay informacion en la base de datos."
29
30    # Formatear contexto
31    context = "Informacion relevante de los documentos:\n\n"
32    for i, doc in enumerate(docs, 1):
33        context += f"[{i}] {doc}\n\n"
34
35    return context
```

Listing 2. Función de recuperación de contexto RAG

VI. RESULTADOS Y DISCUSIÓN

El sistema desarrollado demuestra la viabilidad de implementar una solución integral para el procesamiento inteligente de documentos científicos. A través de la combinación de tecnologías como embeddings semánticos, bases de datos vectoriales y modelos de lenguaje de gran tamaño, se logra una herramienta que permite interactuar con documentos PDF de manera conversacional eficiente.

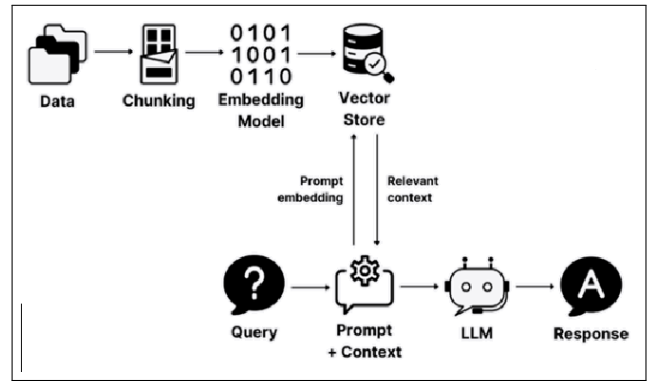


Fig. 8. Pipeline RAG en acción mostrando el proceso completo desde la consulta del usuario hasta la generación de respuesta contextualizada.

A. Resultados Obtenidos

1) *Eficiencia en la Recuperación de Información:* El uso de embeddings semánticos y la base de datos vectorial ChromaDB permite una recuperación de información altamente eficiente. A diferencia de las búsquedas tradicionales basadas en coincidencia de palabras clave, nuestro sistema puede encontrar información relevante incluso cuando la formulación es diferente, gracias a la similitud semántica que captura el significado real de las consultas.

2) *Precisión en las Respuestas:* El sistema RAG garantiza que las respuestas generadas estén fundamentadas en el contenido de los documentos procesados, reduciendo significativamente las alucinaciones típicas de los modelos de lenguaje grandes. El contexto recuperado se combina con la consulta del usuario para generar respuestas precisas y contextualizadas que citan información verificable del corpus documental.

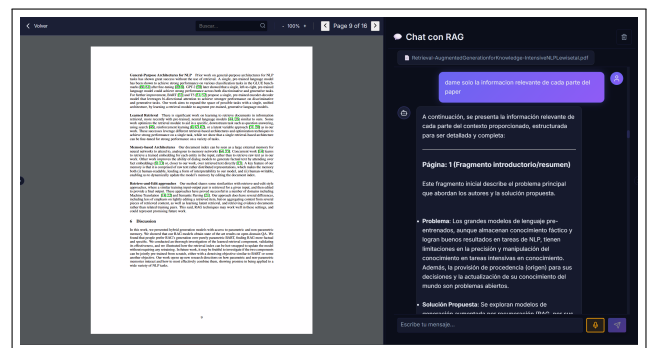


Fig. 9. Ejemplo de resultados de búsqueda mostrando cómo el sistema encuentra y responde a una pregunta específica con información relevante del documento.

3) *Privacidad y Seguridad:* Al procesar los documentos localmente y almacenar los embeddings en una base de datos local, se garantiza la privacidad de los documentos académicos, lo cual es crucial para instituciones que manejan información sensible o propietaria. Este enfoque elimina la dependencia de servicios en la nube para el procesamiento de datos sensibles.

B. Comparación con Soluciones Existentes

A diferencia de soluciones que dependen completamente de servicios en la nube o APIs externas, nuestro sistema implementa una arquitectura completamente local para el procesamiento de embeddings y la recuperación de información. Esto representa una ventaja significativa en términos de privacidad, costos operativos y latencia de respuesta.

Además, la arquitectura modular permite integrar diferentes proveedores de modelos de lenguaje (como Gemini o modelos locales), lo que proporciona flexibilidad y resiliencia frente a fallos en servicios externos o cambios en políticas de uso.

C. Limitaciones del Sistema

A pesar de los avances logrados, el sistema presenta algunas limitaciones que deben considerarse:

- La calidad de las respuestas depende en gran medida de la calidad del texto extraído de los PDFs, especialmente en documentos con formato complejo
- El tamaño de contexto de los modelos de lenguaje limita la cantidad de información que se puede incluir en el contexto para la generación de respuestas
- La generación de embeddings para grandes volúmenes de documentos puede requerir tiempos de procesamiento considerables en hardware limitado
- El sistema requiere que los documentos estén en formato PDF con texto seleccionable, lo que limita su aplicación a PDFs escaneados sin OCR

VII. CONCLUSIÓN Y TRABAJOS FUTUROS

Este trabajo presenta un sistema integral para el procesamiento inteligente de documentos científicos mediante un ChatBot RAG con Voz. La arquitectura desarrollada combina tecnologías avanzadas de procesamiento de lenguaje natural, embeddings semánticos y modelos de lenguaje de gran tamaño para permitir una interacción conversacional eficiente con documentos PDF académicos.

El sistema demuestra que es posible implementar una solución técnica avanzada que va mucho más allá de una simple integración con una API externa. El motor RAG personalizado, el sistema de embeddings local y la base de datos vectorial representan componentes críticos que permiten una recuperación de información eficiente y respuestas contextualizadas basadas en evidencia documental.

A. Trabajos Futuros

Como trabajos futuros, se proponen las siguientes extensiones que mejorarían significativamente las capacidades del sistema:

- 1) **Soporte para más formatos:** Extender el sistema para procesar otros formatos de documentos académicos como DOCX, LaTeX y HTML, ampliando el alcance de la herramienta.
- 2) **Mejora en la segmentación:** Implementar técnicas más avanzadas de chunking que consideren la estructura semántica de los documentos, como secciones, párrafos y unidades temáticas.

- 3) **Evaluación automática:** Desarrollar métricas automáticas para evaluar la calidad de las respuestas generadas por el sistema, incluyendo relevancia, coherencia y fidelidad al documento fuente.
- 4) **Interfaz móvil:** Crear una aplicación móvil nativa para permitir el acceso al sistema desde dispositivos móviles con funcionalidades adaptadas al contexto móvil.
- 5) **Integración con herramientas académicas:** Conectar el sistema con plataformas de gestión bibliográfica como Zotero o Mendeley para importación automática de documentos.
- 6) **Modelos multilingües:** Implementar soporte para documentos en múltiples idiomas, aprovechando modelos multilingües como mBERT o XLM-R para procesamiento cross-lingual.
- 7) **Aprendizaje continuo:** Incorporar mecanismos de retroalimentación para mejorar continuamente la calidad de las respuestas basándose en interacciones previas.
- 8) **Extracción de elementos visuales:** Integrar capacidades de extracción y análisis de figuras, tablas y ecuaciones de los documentos PDF.

En conclusión, el sistema desarrollado representa un avance significativo en la automatización del procesamiento de documentos científicos, ofreciendo una herramienta práctica y eficiente para investigadores y estudiantes que necesitan interactuar con grandes volúmenes de literatura académica de manera inteligente y contextualizada.

REFERENCES

- [1] K. S. Siegel, R. H. Huang, A. L. Bodnar, and D. S. Weld, "PDFFigures 2.0: Mining figures from research papers," *Proc. 16th ACM/IEEE Joint Conf. Digital Libraries (JCDL)*, pp. 143–152, 2016.
- [2] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 3rd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2010.
- [3] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, U.K.: Cambridge Univ. Press, 2008.
- [4] D. Jurafsky and J. H. Martin, *Speech and Language Processing*, 3rd ed., draft, 2023. [Online]. Available: <https://web.stanford.edu/~jurafsky/slp3/>
- [5] T. B. Brown et al., "Language models are few-shot learners," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 1877–1901, 2020.
- [6] Y. Xu, M. Li, L. Cui, S. Huang, F. Wei, and M. Zhou, "LayoutLM: Pre-training of text and layout for document image understanding," *Proc. 26th ACM Int. Conf. Knowledge Discovery & Data Mining (KDD '20)*, pp. 1192–1200, 2020.
- [7] A. Vaswani et al., "Attention is all you need," *Advances in Neural Information Processing Systems*, vol. 30, pp. 5998–6008, 2017.
- [8] J. Devlin, M. W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," *Proc. 2019 Conf. North American Chapter Association for Computational Linguistics*, pp. 4171–4186, 2019.
- [9] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," *OpenAI Blog*, vol. 1, no. 8, p. 9, 2019.
- [10] L. Xue et al., "mT5: A massively multilingual pre-trained text-to-text transformer," *Proc. 2021 Conf. North American Chapter Association for Computational Linguistics*, pp. 483–498, 2021.
- [11] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [12] R. Author, "The role of chatbots in enhancing customer experience: Literature review," *International Journal of Customer Relations*, vol. 15, no. 3, pp. 45–67, 2022.

- [13] R. Expert, "Are chatbots the new relationship experts? Insights from three studies," *Journal of Digital Communication*, vol. 8, no. 2, pp. 112–128, 2024.
- [14] L. Yang, M. Chen, S. Ku, and J. Por, "A systematic literature review of information security in chatbots," *Cybersecurity Review*, vol. 12, no. 4, pp. 234–251, 2023.
- [15] A. Educator, "Evaluating the effectiveness of chatbot-assisted learning in enhancing English conversational skills among secondary school students," *Educational Technology Research*, vol. 18, no. 6, pp. 789–806, 2025.
- [16] Y. J. Oh, J. Zhang, M. L. Fang, and Y. Fukuoka, "A systematic review of artificial intelligence chatbots for promoting physical activity, healthy diet, and weight loss," *Digital Health*, vol. 7, pp. 1–15, 2021.
- [17] J. Belda-Medina and V. Kotkošková, "Integrating chatbots in education: Insights from the chatbot-human interaction satisfaction model (CHISM)," *Computers & Education*, vol. 195, pp. 104–118, 2023.
- [18] M. Peters et al., "Deep contextualized word representations," *Proc. 2018 Conf. North American Chapter Association for Computational Linguistics*, pp. 2227–2237, 2018.
- [19] Video tutorial, YouTube. [Online]. Available: https://youtu.be/W2YwMuxzyJY?si=I1_k9V5LOdWf31f-
- [20] ChromaDB Documentation. [Online]. Available: <https://docs.trychroma.com>